

Übung 3: Klassen, Objekte und Referenzen

Aufgabe 0: Einführung	1
Aufgabe 1: Bankkonto	2
Aufgabe 2: Information Hiding	2
Aufgabe 3: Banksystem	3
Aufgabe 4: Methoden	4
Aufgabe 5: Referenzen (Theorie)	4
Aufgabe 6: Rechtecke (fakultativ)	6

Lernziele

- Klassen entwerfen und implementieren.
- Objekte instanziiieren und verwenden.
- Referenzen verstehen und Objektstrukturen erstellen.

Aufgabe 0: Einführung

Zum Einstieg schauen wir uns folgenden Code an:

```
1 class Rectangle {
2     int length;
3     int width;
4
5     void stretch(int factor) {
6         length *= this.factor;
7         width *= this.factor;
8     }
9
10    static int getArea() {
11        return this.length * this.width;
12    }
13 }
```

Der Code enthält zwei Fehler, finden Sie diese ohne Ihre IDE/Compiler?

Aufgabe 1: Bankkonto

Für ein Banksystem sollen die Bankkonten als Objekte realisiert werden. Entwickeln Sie dazu eine Klasse `BankAccount` mit folgenden Eigenschaften:

- Zustand mit Instanzvariablen:
 - **Kontonummer** (Typ `long`, Variablenname wählbar)
 - **Kontostand** (Typ `double` Variablenname wählbar)
- Verhalten mit Instanzmethoden:
 - **`deposit()`**: Ermöglicht das Einzahlen eines Geldbetrages auf das Konto. Falls ein negativer Geldbetrag einzuzahlen versucht wird, ändert sich der Kontostand nicht und es wird lediglich `false` zurückgegeben.
 - **`withdraw()`**: Erlaubt das Abheben eines Geldbetrages vom Konto, sofern der Kontostand dabei nicht negativ wird. Ansonsten wird nichts abgehoben und `false` zurückgegeben.
- Konstruktion:
 - Bei Eröffnung eines Kontos (Erzeugung des `BankAccount`-Objekts) wird die Kontonummer benötigt und muss mitgegeben werden. Zudem soll der Kontostand mit dem Wert 0 initialisiert werden.

Prüfen Sie die Arbeitsweise Ihrer Klasse mit ein paar Objekten und einem kleinen Szenario (Geld einzahlen und abheben).

Aufgabe 2: Information Hiding

Verbessern Sie das Design Ihrer `BankAccount`-Klasse aus Aufgabe 1 wie folgt:

- Die Kontonummer eines Bankkontos soll nach der Konstruktion unveränderlich sein und nur gelesen werden können.
- Der Kontostand soll nur über die Methoden `deposit()` und `withdraw()` verändert werden.

Tipp: Verwenden Sie dazu die Sichtbarkeitsmodifikatoren (`private` und `public`). Denken Sie auch an das Schlüsselwort `final`.

Aufgabe 3: Banksystem

Das Banksystem wird nun weiter ausgebaut: Neu werden die Klassen **BankCustomer** und **BankManager** mit folgenden Eigenschaften eingeführt:

- Ein **BankCustomer** hat einen Vornamen, Nachnamen, eine Adresse und ein Alter.
- Ein **BankManager** hat einen Namen und eine interne Manager-ID (**int**) der Bank.

Zusätzlich gibt es folgende Beziehungen:

- Jeder **BankAccount** hat eine Referenz zum **BankCustomer** (der Eigentümer des Kontos)
- Jeder **BankCustomer** hat eine Referenz zum **BankManager** (der Betreuer des Kunden)

Grafisch können Sie sich die Beziehungen zwischen den Klassen wie folgt vorstellen:



Figure 1: Beziehungen zwischen den Klassen

Bitte beachten Sie, dass ein Kunde mehrere Konten besitzen kann und ein Manager mehrere Kunden betreuen darf.

Überprüfen Sie die Funktionsweise Ihres neuen Programms, indem Sie die folgende Objektstruktur (bestehend aus Instanzen und Referenzen) aufbauen.

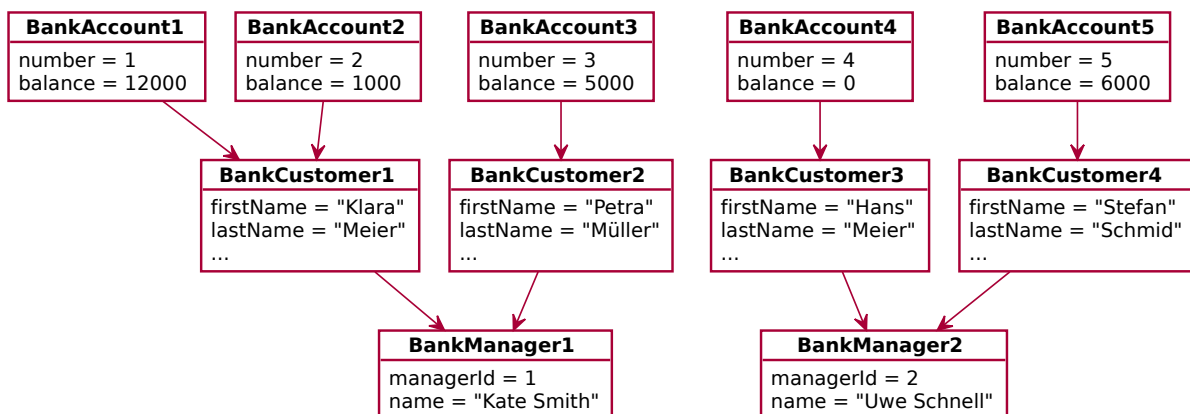


Figure 2: Objektstruktur mit Referenzen

Aufgabe 4: Methoden

Implementieren Sie für Ihre Bank-Lösung aus der vorangehenden Aufgabe zusätzlich diese Methoden:

Für alle Klassen:

- `void print()`
 - Gibt Informationen über das Objekt auf der Konsole aus.

Für Klasse `BankAccount`:

- `boolean sameCustomer(BankAccount other)`
 - Prüft, ob das Konto `other` vom selben Kunden wie dieses Konto ist.
- `BankManager getManager()`
 - Gibt den Manager des Kunden dieses Kontos zurück.

Für Klasse `BankCustomer`:

- `BankAccount openNewAccount(long number)`
 - Erzeugt ein neues Konto mit der angegebenen Kontonummer für den Kunden und gibt es zurück.

Aufgabe 5: Referenzen (Theorie)

Die folgende Klasse dient als Ausgangsbasis:

```
1 class Person {
2     String name;
3     Person father, mother;
4     Person[] colleagues;
5
6     public Person(String name) {
7         this.name = name;
8     }
9 }
```

Bestimmen Sie die Ausgaben der folgenden Code-Abschnitte. Versuchen Sie es zuerst ohne Computer zu lösen, indem Sie schrittweise die Objekte mit den Referenzen aufzeichnen. Überprüfen Sie ihre Lösung danach mit einem Programm und allenfalls mit dem Debugger.

a)

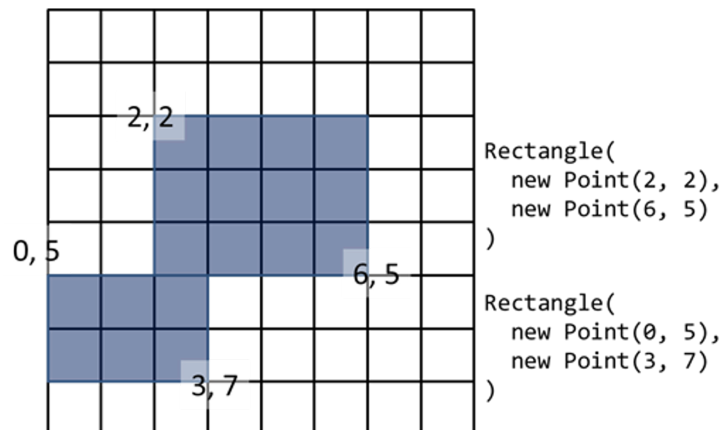
```
1 void main()
2 {
3     Person a = new Person("Claudia");
4     Person b = new Person("Peter");
5     a.father = b;
6     b.name = "Pesche";
7     b = new Person("Susanne");
8     a.mother = b;
9     b.father = new Person("Albert");
10    b = null;
11
12    IO.println(a.father.name);
13    IO.println(a.mother.name);
14    IO.println(a.mother.father.name);
15    IO.println(a.father.mother.name);
16 }
```

b)

```
1 void main()
2 {
3     Person a = new Person("Felix");
4     Person b = new Person("Regula");
5     a.colleagues = new Person[2];
6     a.colleagues[0] = b;
7     a.colleagues[1] = new Person("Beat");
8     b.colleagues = new Person[2];
9     b.colleagues[0] = a;
10    b.colleagues[1] = a.colleagues[1];
11    b.colleagues[1].name = "Beatus";
12
13    a = a.colleagues[0];
14    IO.println(a.name);
15
16    a = a.colleagues[0];
17    IO.println(a.name);
18
19    a = a.colleagues[1];
20    IO.println(a.name);
21
22    a.colleagues = b.colleagues;
23    a = a.colleagues[0];
24    IO.println(a.name);
25 }
```

Aufgabe 6: Rechtecke (fakultativ)

Es werden Rechtecke in einem ganzzahligen 2D-Raster angeordnet. Die Rechtecke sind durch zwei Punkte definiert: den oberen linken Punkt sowie den unteren rechten Punkt.



Schreiben Sie eine Klasse `Rectangle`, die dies modelliert. Die zwei Eckpunkte eines Rechtecks sollen als Instanzen einer Klasse `Point` verwaltet werden.

Die Klasse `Point` soll folgende Konstruktoren/Methoden bieten:

- `Point(int x, int y)`
 - Initialisiert einen Punkt mit Angabe der x- und y-Koordinate.
- `int getX()`
 - Liefert die x-Koordinate zurück.
- `int getY()`
 - Liefert die y-Koordinate zurück.
- `boolean isSame(Point other)`
 - Prüft, ob dieser Punkt wertgleich zu einem anderen Punkt ist, also konkret, ob beide die gleichen Werte für x- und y-Koordinate besitzen.

Die Klasse `Rectangle` soll folgende Konstruktoren/Methoden besitzen:

- `Rectangle(Point topLeft, Point bottomRight)`
 - Initialisiert ein Rechteck mit Angabe des linken oberen Punkts und des rechten unteren Punkts.
- `Rectangle(Point topLeft, int size)`
 - Initialisiert ein Quadrat mit Angabe des linken oberen Punkts und der Seitenlänge.
- `getTopLeft()`
 - Getter auf den Punkt `topLeft`.
- `getBottomRight()`
 - Getter auf Punkt `bottomRight`.
- `boolean isSquare()`
 - Prüft, ob das Rechteck ein Quadrat ist.
- `boolean isSame(Rectangle other)`
 - Prüft, ob dieses Rechteck gleich ist wie ein anderes (gleicher Ort und gleiche Grösse).
- `boolean encloses(Rectangle other)`
 - Prüft, ob dieses Rechteck ein anderes beinhaltet (oder genau gleich ist).
- `boolean overlaps(Rectangle other)`
 - Prüft, ob dieses und ein anderes Rechteck sich überlappen. Dabei gilt: Nur Berühren am Rand gilt noch nicht als Überlappen.
- `Rectangle stretch(int factor)`
 - Gibt ein neues Rechteck zurück, das die Länge und Breite mit einem Faktor gestreckt hat. Dabei verändert sich die obere linke Ecke nicht!
- `Rectangle shrink(int factor)`
 - Gibt ein neues Rechteck zurück, das die Länge und Breite um einen Faktor gestaucht wurde. Dabei verändert sich die obere linke Ecke nicht!

Überprüfen Sie Ihre Klasse mit einem Szenario aus ein paar Rechteck-Exemplaren. In der Vorlage finden Sie bereits Unit Tests für `Point` und `Rectangle`. Diese funktionieren initial noch nicht korrekt, weil die Klassen noch nicht über Methoden mit den richtigen Implementierungen verfügen.

Um alle Tests auszuführen, klicken Sie rechts auf das `src/test/java`-Verzeichnis und wählen Sie «Run 'All Test'» (IntelliJ IDEA / «Run As -> JUnit Test» (Eclipse).

Hinweis: Die Methoden `stretch()` und `shrink()` sollen ein sogenanntes “Method-Chaining” ermöglichen, Beispiel:

```
1 Rectangle rect = new Rectangle(new Point(1, 1), new Point(5, 9));
2 Rectangle rect2 = new Rectangle(new Point(1, 1), new Point(5, 9));
3
4 rect.stretch(2).shrink(2).isSame(rect2)
```

(Quelle: Aufgabe in Anlehnung an R. Schiedermeier. Programmieren mit Java, 2010.)